

ning-with-explainable-ai-xai-shap

March 1, 2024

```
[ ]: import numpy as np
import pandas as pd
import os

from google.colab import drive
drive.mount('/content/drive')
```

COPYRIGHT OFFICE
NEW DELHI
Reg. No. - SW-18921/2024
Date 03/06/2024

```
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-2-018dc4f88127> in <cell line: 6>()
      4
      5 from google.colab import drive
----> 6 drive.mount('/content/drive')

/usr/local/lib/python3.10/dist-packages/google/colab/drive.py in _
    ↪ mount(mountpoint, force_remount, timeout_ms, readonly)
      98 def mount(mountpoint, force_remount=False, timeout_ms=120000,
    ↪ readonly=False):
      99     """Mount your Google Drive at the specified mountpoint path."""
--> 100     return _mount(
      101         mountpoint,
      102         force_remount=force_remount,

/usr/local/lib/python3.10/dist-packages/google/colab/drive.py in _
    ↪ _mount(mountpoint, force_remount, timeout_ms, ephemeral, readonly)
      127 )
      128 if ephemeral:
--> 129     _message.blocking_request(
      130         'request_auth', request={'authType': 'dfs_ephemeral'},
    ↪ timeout_sec=None
      131 )

/usr/local/lib/python3.10/dist-packages/google/colab/_message.py in _
    ↪ _blocking_request(request_type, request, timeout_sec, parent)
      132     request_type, request, parent=parent, expect_reply=True
    ↪ )
      133 return read_reply_from_input(request_id, timeout_sec)
```



Dr. Anil
उन्नत की ओर

```

/usr/local/lib/python3.10/dist-packages/google/colab/_message.py in
↳ read_reply_from_input(message_id, timeout_sec)
    94     reply = _read_next_input_message()
    95     if reply == _NOT_READY or not isinstance(reply, dict):
--> 96         time.sleep(0.025)
    97         continue
    98     if (

```

KeyboardInterrupt:

```

[ ]: import os
from sklearn.model_selection import train_test_split
import random

drive_path = '/content/drive/MyDrive/brain-tumor-mri-dataset'
all_images = []

for data_type in ['Training', 'Testing']:
    for tumor_type in ['notumor', 'pituitary', 'glioma', 'meningioma']:
        class_path = os.path.join(drive_path, data_type, tumor_type)
        class_images = [os.path.join(class_path, img) for img in os.
↳ listdir(class_path)]
        all_images.extend(class_images)

labels = [os.path.dirname(img).split(os.path.sep)[-1] for img in all_images]

# X_train, X_test, Y_train, Y_test = train_test_split(
#     all_images, labels, test_size=0.2, random_state=42, stratify=labels
# )

# print("Number of Training Samples:", len(X_train))
# print("Number of Testing Samples:", len(X_test))

train_images, test_images, train_labels, test_labels = train_test_split(
    all_images, labels, test_size=0.2, random_state=42, stratify=labels
)

# train_images_selected = random.sample(train_images, k=2400)
# test_images_selected = random.sample(test_images, k=600)

def extract_labels(image_paths):
    return [os.path.dirname(img).split(os.path.sep)[-1] for img in image_paths]

```

```

= train_images
= extract_labels(X_train)

```

Dr. Anil
डॉ. अनिल की ओडिशा

```
X_test = test_images
Y_test = extract_labels(X_test)

print("Selected Training Images:", len(X_train))
print("Training Labels:", len(Y_train))
print("Selected Testing Images:", len(X_test))
print("Testing Labels:", len(Y_test))
```

Selected Training Images: 5618
 Training Labels: 5618
 Selected Testing Images: 1405
 Testing Labels: 1405

```
[ ]: import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array
def preprocess_images(image_paths, target_size=(224, 224)):
    images = []
    for img_path in image_paths:
        img = load_img(img_path, target_size=target_size)
        img_array = img_to_array(img)
        images.append(img_array)
    return np.array(images, dtype="float16")
```

```
[ ]: import tensorflow as tf
```

```
[ ]: tf.test.gpu_device_name()
```

```
[ ]: '/device:GPU:0'
```

```
[ ]: from sklearn.preprocessing import LabelEncoder

X_train_processed = preprocess_images(X_train)
X_test_processed = preprocess_images(X_test)

label_encoder = LabelEncoder()
Y_train = label_encoder.fit_transform(Y_train)
Y_test = label_encoder.transform(Y_test)

Y_train = np.array(Y_train)
Y_test = np.array(Y_test)
```

```
[ ]: import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam
```



Dr. Anil
 इकाई की ओर

```
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

for layer in base_model.layers:
    layer.trainable = False

model = models.Sequential()
model.add(base_model)
model.add(layers.GlobalAveragePooling2D())
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(4, activation='softmax'))
```

COPYRIGHT OFFICE
NEW DELHI
Reg. No. - 50/18921/2024
Date 03/06/2024

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94/65/36/94/65736 [=====] - 5s 0us/step

```
[ ]: model.compile(optimizer=Adam(learning_rate=0.0001),
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])
```

```
[ ]: history = model.fit(X_train_processed, Y_train, epochs=20,
                        validation_data=(X_test_processed, Y_test))
```

```
Epoch 1/20
176/176 [=====] - 37s 165ms/step - loss: 0.7407 -
accuracy: 0.7147 - val_loss: 0.3426 - val_accuracy: 0.8918
Epoch 2/20
176/176 [=====] - 21s 121ms/step - loss: 0.3998 -
accuracy: 0.8494 - val_loss: 0.2674 - val_accuracy: 0.9075
Epoch 3/20
176/176 [=====] - 21s 121ms/step - loss: 0.3173 -
accuracy: 0.8827 - val_loss: 0.2435 - val_accuracy: 0.9110
Epoch 4/20
176/176 [=====] - 21s 121ms/step - loss: 0.2679 -
accuracy: 0.9060 - val_loss: 0.2093 - val_accuracy: 0.9224
Epoch 5/20
176/176 [=====] - 21s 122ms/step - loss: 0.2415 -
accuracy: 0.9122 - val_loss: 0.1912 - val_accuracy: 0.9246
Epoch 6/20
176/176 [=====] - 21s 121ms/step - loss: 0.2173 -
accuracy: 0.9227 - val_loss: 0.1760 - val_accuracy: 0.9302
Epoch 7/20
[=====] - 21s 121ms/step - loss: 0.1939 -
: 0.9322 - val_loss: 0.1696 - val_accuracy: 0.9345
20
```



Dr. Anil
उन्नत की ओर

176/176 [=====] - 21s 121ms/step - loss: 0.1778 - accuracy: 0.9356 - val_loss: 0.1583 - val_accuracy: 0.9423

Epoch 9/20

176/176 [=====] - 21s 121ms/step - loss: 0.1679 - accuracy: 0.9422 - val_loss: 0.1561 - val_accuracy: 0.9452

Epoch 10/20

176/176 [=====] - 21s 122ms/step - loss: 0.1477 - accuracy: 0.9477 - val_loss: 0.1488 - val_accuracy: 0.9402

Epoch 11/20

176/176 [=====] - 21s 122ms/step - loss: 0.1463 - accuracy: 0.9496 - val_loss: 0.1416 - val_accuracy: 0.9473

Epoch 12/20

176/176 [=====] - 21s 122ms/step - loss: 0.1276 - accuracy: 0.9566 - val_loss: 0.1334 - val_accuracy: 0.9466

Epoch 13/20

176/176 [=====] - 21s 121ms/step - loss: 0.1261 - accuracy: 0.9537 - val_loss: 0.1267 - val_accuracy: 0.9502

Epoch 14/20

176/176 [=====] - 21s 121ms/step - loss: 0.1147 - accuracy: 0.9619 - val_loss: 0.1237 - val_accuracy: 0.9523

Epoch 15/20

176/176 [=====] - 21s 121ms/step - loss: 0.1061 - accuracy: 0.9605 - val_loss: 0.1213 - val_accuracy: 0.9559

Epoch 16/20

176/176 [=====] - 21s 121ms/step - loss: 0.0958 - accuracy: 0.9685 - val_loss: 0.1249 - val_accuracy: 0.9523

Epoch 17/20

176/176 [=====] - 21s 122ms/step - loss: 0.0939 - accuracy: 0.9715 - val_loss: 0.1123 - val_accuracy: 0.9544

Epoch 18/20

176/176 [=====] - 21s 121ms/step - loss: 0.0867 - accuracy: 0.9728 - val_loss: 0.1198 - val_accuracy: 0.9580

Epoch 19/20

176/176 [=====] - 21s 121ms/step - loss: 0.0835 - accuracy: 0.9735 - val_loss: 0.1072 - val_accuracy: 0.9616

Epoch 20/20

176/176 [=====] - 21s 121ms/step - loss: 0.0775 - accuracy: 0.9756 - val_loss: 0.1130 - val_accuracy: 0.9544

```
[ ]: test_loss, test_acc = model.evaluate(X_test_processed, Y_test)
print(f'Test accuracy: {test_acc}')
```

44/44 [=====] - 4s 95ms/step - loss: 0.1130 - accuracy: 0.9544

Test accuracy: 0.954448401927948



Dr. Anil
उन्नत की ओर

```
[ ]: predicted_labels = model.predict(X_test_processed).argmax(axis=1)

misclassified_indices = []

for i in range(len(Y_test)):
    if predicted_labels[i] != Y_test[i]:
        misclassified_indices.append(i)
print(misclassified_indices)
```

COPYRIGHT NOTICE
 NEW DELHI
 Reg. No. - 424, 433, 463,
 Date 03/05/2024
 694 705 718

```
[ ]: !pip install shap --upgrade
```

```
Downloading shap-0.44.0-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64
.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (533 kB)
```

```
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.10/dist-
 (from shap) (4.66.1)
```



Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.10/dist-packages (from shap) (23.2)

Collecting slicer==0.0.7 (from shap)

Downloading slicer-0.0.7-py3-none-any.whl (14 kB)

Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from shap) (0.58.1)

Requirement already satisfied: cloudpickle in /usr/local/lib/python3.10/dist-packages (from shap) (2.2.1)

Requirement already satisfied: llvmlite<0.42,>=0.41.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba->shap) (0.41.1)

Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2023.3.post1)

Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (1.3.2)

Requirement already satisfied: threadpoolctl>=2.0.0 in

/usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (3.2.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.1->pandas->shap) (1.16.0)

Installing collected packages: slicer, shap

Successfully installed shap-0.44.0 slicer-0.0.7

COPYRIGHT
NEW DELHI
Reg. No. - SW/19821/2024
Date 03/06/2024

```
[ ]: import shap
```

```
## Define a masker using a blurred background  
masker = shap.maskers.Image("inpaint_telea", (224,224,3))
```

```
[ ]: def original_loss(Y_test, predicted_labels):  
    original_loss = tf.keras.losses.sparse_categorical_crossentropy(Y_test,  
        predicted_labels, from_logits=True)  
    return original_loss
```

```
[ ]: shap_val=0  
# def wrapper_function(shap_values, lambda_):  
#     def custom_loss(Y_test, predicted_labels):  
#         original_loss = tf.keras.losses.  
#         sparse_categorical_crossentropy(Y_test, predicted_labels, from_logits=True)  
#         shap_val=tf.cast(shap_values.values, tf.float32)  
#         regularization_term = lambda_ * tf.reduce_sum(tf.abs(shap_val))  
#         total_loss = original_loss + regularization_term  
#         return total_loss  
#     return custom_loss  
- -  
om_loss_with_weights(Y_test, predicted_labels, sample_weights):  
;inal_loss = tf.keras.losses.sparse_categorical_crossentropy(Y_test,  
cted_labels, from_logits=True)
```



Dr. Anil
इकांत की मंडित

```
sample_weights = tf.cast(tf.reshape(sample_weights, (-1, 1)), tf.float32)
weighted_loss = tf.reduce_mean(tf.multiply(sample_weights, original_loss))
return weighted_loss
```

```
[ ]: accuracy_threshold = 0.99

while True:
    explainer = shap.Explainer(model, masker, output_names=["notumor",
    ↪ "pituitary", "glioma", "meningioma"])
    misclassified_images = X_test_processed[misclassified_indices]
    shap_values = explainer(misclassified_images, max_evals=500, batch_size=16)

    retrained_model = models.clone_model(model)
    retrained_model.set_weights(model.get_weights())

    if iteration > 0:
        retrained_model.set_weights(previous_retrained_model.get_weights())

    for channel_index in range(X_train_processed.shape[-1]):
        shap_values_channel = shap_values[..., channel_index].values
        base_shap_values_channel = shap_values[..., channel_index].base_values

        positive_shap_mask = base_shap_values_channel > 0
        positive_shap_positions = np.where(positive_shap_mask)

        # shap_values_flat = shap_values_channel.flatten()
        # shap_values_resaped = np.reshape(shap_values_flat,
        ↪ base_shap_values_channel.shape)

        shap_values_difference = shap_values_channel -
        ↪ base_shap_values_channel[:, np.newaxis, np.newaxis, np.newaxis]

        delta_weight = np.mean(shap_values_difference[positive_shap_positions])
        delta_weight = tf.cast(delta_weight, tf.float32)

        sample_weights = np.ones_like(Y_train)
        sample_weights[Y_train == channel_index] += delta_weight

        X_train_appended = np.concatenate((X_train_processed,
        ↪ X_test_processed[misclassified_indices]))
        Y_train_appended = np.concatenate((Y_train, Y_test[misclassified_indices]))

        retrained_model.compile(optimizer=Adam(learning_rate=0.0001), loss=lambda
        ↪ true labels, predicted_labels: custom_loss_with_weights(true_labels,
        ctected_labels, sample_weights), metrics=['accuracy'])
```

COPYRIGHT OFFICE
NEW DELHI
 Reg. No. - SW-18921/2024
 Date 03/05/2024



Dr. Anil
 कुशल की ओर

```

retrained_model.fit(X_train_appended, Y_train_appended, epochs=10,
↪validation_data=(X_test_processed, Y_test))

iteration += 1

test_loss, test_accuracy = retrained_model.evaluate(X_test_processed,
↪Y_test)

predicted_labels = retrained_model.predict(X_test_processed).argmax(axis=1)

misclassified_indices = [i for i in range(len(Y_test)) if
↪predicted_labels[i] != Y_test[i]]

iteration_list.append(iteration)
accuracy_list.append(test_accuracy)
misclassified_list.append(len(misclassified_indices))

previous_retrained_model = retrained_model

print(f"Iteration {iteration} - Test Accuracy: {test_accuracy}")

with open(f'Iteration_{iteration}_model.pkl', 'wb') as file:
    pickle.dump(retrained_model, file)

if test_accuracy >= accuracy_threshold:
    print(f"Accuracy reached the threshold ({accuracy_threshold}). Stopping
↪training.")
    break

```

```

0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  2%|          | 1/64 [00:00<?, ?it/s]
0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  5%|          | 3/64 [00:43<10:06,  9.95s/it]
0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  6%|          | 4/64 [01:02<13:41, 13.69s/it]
0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  8%|          | 5/64 [01:22<15:46, 16.04s/it]
0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  9%|          | 6/64 [01:41<16:32, 17.12s/it]
          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer: 11%|          | 7/64 [02:00<16:54, 17.81s/it]

```



Dr. Anil
उन्नत की ओर

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 12%| | 8/64 [02:21<17:26, 18.68s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 14%| | 9/64 [02:40<17:15, 18.82s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 16%| | 10/64 [02:59<17:03, 18.95s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 17%| | 11/64 [03:20<17:13, 19.50s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 19%| | 12/64 [03:39<16:57, 19.57s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 20%| | 13/64 [04:01<17:07, 20.14s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 22%| | 14/64 [04:20<16:25, 19.71s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 23%| | 15/64 [04:37<15:27, 18.93s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 25%| | 16/64 [04:59<15:57, 19.95s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 27%| | 17/64 [05:19<15:37, 19.96s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 28%| | 18/64 [05:39<15:12, 19.84s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 30%| | 19/64 [05:59<14:56, 19.92s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 31%| | 20/64 [06:17<14:19, 19.53s/it]

0%| | 0/498 [00:00<?, ?it/s]

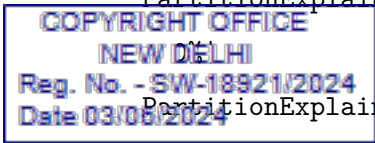
PartitionExplainer explainer: 33%| | 21/64 [06:36<13:51, 19.35s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 34%| | 22/64 [06:56<13:32, 19.35s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 36%| | 23/64 [07:18<13:45, 20.14s/it]



Explainer explainer: 36%|

Dr. Anil
उत्कृष्ट की मंडल

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 38%| | 24/64 [07:36<13:01, 19.55s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 39%| | 25/64 [07:57<12:59, 19.99s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 41%| | 26/64 [08:15<12:18, 19.42s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 42%| | 27/64 [08:36<12:23, 20.09s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 44%| | 28/64 [08:56<11:52, 19.80s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 45%| | 29/64 [09:14<11:20, 19.45s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 47%| | 30/64 [09:37<11:37, 20.51s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 48%| | 31/64 [09:56<10:56, 19.89s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 50%| | 32/64 [10:15<10:28, 19.63s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 52%| | 33/64 [10:34<10:07, 19.61s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 53%| | 34/64 [10:53<09:43, 19.44s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 55%| | 35/64 [11:14<09:32, 19.75s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 56%| | 36/64 [11:31<08:54, 19.09s/it]

0%| | 0/498 [00:00<?, ?it/s]

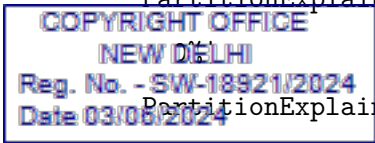
PartitionExplainer explainer: 58%| | 37/64 [11:50<08:30, 18.92s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 59%| | 38/64 [12:08<08:07, 18.75s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 61%| | 39/64 [12:27<07:47, 18.68s/it]



Explainer explainer: 61%| | 39/64 [12:27<07:47, 18.68s/it]

Dr. Anil
उत्कृष्ट की अंशित

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 62%| | 40/64 [12:46<07:33, 18.89s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 64%| | 41/64 [13:06<07:20, 19.15s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 66%| | 42/64 [13:28<07:18, 19.92s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 67%| | 43/64 [13:49<07:09, 20.44s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 69%| | 44/64 [14:09<06:44, 20.22s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 70%| | 45/64 [14:28<06:16, 19.83s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 72%| | 46/64 [14:47<05:54, 19.68s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 73%| | 47/64 [15:07<05:37, 19.86s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 75%| | 48/64 [15:27<05:14, 19.66s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 77%| | 49/64 [15:50<05:09, 20.63s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 78%| | 50/64 [16:09<04:44, 20.32s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 80%| | 51/64 [16:29<04:22, 20.22s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 81%| | 52/64 [16:48<03:57, 19.81s/it]

0%| | 0/498 [00:00<?, ?it/s]

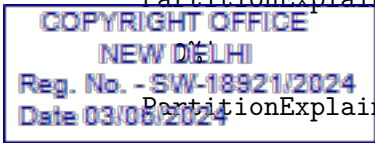
PartitionExplainer explainer: 83%| | 53/64 [17:07<03:35, 19.56s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 84%| | 54/64 [17:28<03:20, 20.04s/it]

0%| | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 86%| | 55/64 [17:46<02:53, 19.30s/it]



Dr. Anil
उत्कृष्ट की ओर

0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 88%| | 56/64 [18:03<02:30, 18.81s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 89%| | 57/64 [18:22<02:12, 18.89s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 91%| | 58/64 [18:42<01:54, 19.03s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 92%| | 59/64 [19:00<01:33, 18.76s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 94%| | 60/64 [19:21<01:17, 19.38s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 95%| | 61/64 [19:39<00:56, 18.98s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 97%| | 62/64 [19:58<00:38, 19.17s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 98%| | 63/64 [20:18<00:19, 19.33s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 100%| | 64/64 [20:37<00:00, 19.30s/it]
 0%| | 0/498 [00:00<?, ?it/s]
 PartitionExplainer explainer: 65it [20:54, 19.61s/it]

Epoch 1/10

"`sparse\_categorical\_crossentropy` received `from\_logits=True`, but the `output` argument was produced by a Softmax activation and thus does not represent logits. Was this intended?

178/178 [=====] - 29s 144ms/step - loss: 0.0638 - accuracy: 0.9688 - val_loss: 0.0682 - val_accuracy: 0.9623

Epoch 2/10

178/178 [=====] - 22s 122ms/step - loss: 0.0649 - accuracy: 0.9680 - val_loss: 0.0670 - val_accuracy: 0.9623

Epoch 3/10

178/178 [=====] - 21s 121ms/step - loss: 0.0584 - accuracy: 0.9720 - val_loss: 0.0537 - val_accuracy: 0.9730

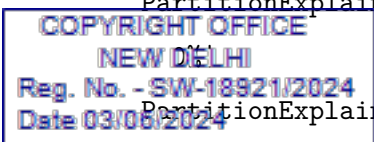
Epoch 4/10

[=====] - 22s 121ms/step - loss: 0.0511 - accuracy: 0.9775 - val_loss: 0.0591 - val_accuracy: 0.9644

10

[=====] - 22s 122ms/step - loss: 0.0496 -

Dr. Anil
 उन्नत की ओर



accuracy: 0.9778 - val_loss: 0.0438 - val_accuracy: 0.9815
Epoch 6/10
178/178 [=====] - 22s 121ms/step - loss: 0.0424 -
accuracy: 0.9815 - val_loss: 0.0473 - val_accuracy: 0.9744
Epoch 7/10
178/178 [=====] - 22s 121ms/step - loss: 0.0422 -
accuracy: 0.9833 - val_loss: 0.0407 - val_accuracy: 0.9829
Epoch 8/10
178/178 [=====] - 22s 121ms/step - loss: 0.0390 -
accuracy: 0.9820 - val_loss: 0.0378 - val_accuracy: 0.9836
Epoch 9/10
178/178 [=====] - 22s 121ms/step - loss: 0.0345 -
accuracy: 0.9863 - val_loss: 0.0340 - val_accuracy: 0.9851
Epoch 10/10
178/178 [=====] - 22s 121ms/step - loss: 0.0356 -
accuracy: 0.9852 - val_loss: 0.0295 - val_accuracy: 0.9922
44/44 [=====] - 4s 97ms/step - loss: 0.0295 - accuracy:
0.9922
44/44 [=====] - 5s 94ms/step
Iteration 1 - Test Accuracy: 0.9921708106994629

The object being serialized includes a `lambda`. This is unsafe. In order to reload the object, you will have to pass `safe\_mode=False` to the loading function. Please avoid using `lambda` in the future, and use named Python functions instead. This is the `lambda` being serialized:
retrained_model.compile(optimizer=Adam(learning_rate=0.0001),loss=lambda true_labels, predicted_labels: custom_loss_with_weights(true_labels, predicted_labels, sample_weights), metrics=['accuracy'])

Accuracy reached the threshold (0.99). Stopping training.

```
[ ]: import os
from sklearn.model_selection import train_test_split
import random

drive_path = '/content/drive/MyDrive/brain-tumor-mri-dataset'

all_images = []

for data_type in ['Training', 'Testing']:
    for tumor_type in ['notumor', 'pituitary', 'glioma', 'meningioma']:
        class_path = os.path.join(drive_path, data_type, tumor_type)
        class_images = [os.path.join(class_path, img) for img in os.
↳listdir(class_path)]
        all_images.extend(class_images)

[os.path.dirname(img).split(os.path.sep)[-1] for img in all_images]
```



Dr. Ankit
डॉ. अंकित की ओडिओ

```

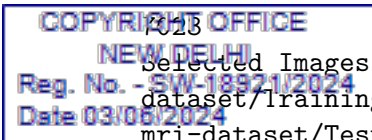
train_images, test_images, train_labels, test_labels = train_test_split(
    all_images, labels, test_size=0.2, random_state=42, stratify=labels
)

print(len(set(all_images)))
print(len(set(train_images + test_images)))
# Select 100 random images not in X_train and X_test
selected_images = random.sample(set(all_images) - set(train_images_selected +
↪test_images_selected), k=100)

print("Selected Images:", selected_images)

```

7023

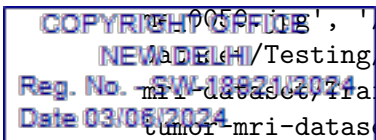


Selected Images: ['/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0250.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/pituitary/Te-pi_0072.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_1085.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_1065.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0497.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0185.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1277.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0358.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/glioma/Te-gl_0192.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0264.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0588.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0475.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/glioma/Te-gl_0220.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0501.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_1056.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0161.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0277.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0350.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1204.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0538.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0728.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0988.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0051.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0364.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0398.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0149.jpg',



Dr. Anil
इकांत की ओड़ि

'/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0503.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0983.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0794.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0412.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0392.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0989.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0835.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_1096.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/meningioma/Te-me_0046.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/pituitary/Te-pi_0028.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_1148.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/meningioma/Te-me_0059.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/pituitary/Te-pi_0109.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_1290.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0386.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1202.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0884.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1070.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/pituitary/Te-pi_0067.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0129.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_1253.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1365.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_1077.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0488.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1453.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_1053.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0448.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0043.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0171.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_1031.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0149.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_1026.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0165.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0098.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0366.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_1332.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0524.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0861.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0861.jpg'



डस्ता की मंडिर

tumor-mri-dataset/Training/pituitary/Tr-pi_1063.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1023.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0299.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0020.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_1136.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0019.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0289.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0122.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0231.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/pituitary/Tr-pi_0236.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0272.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/glioma/Te-gl_0180.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0499.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1190.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0446.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0225.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1293.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0341.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0913.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0447.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0821.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/glioma/Te-glTr_0008.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/meningioma/Te-me_0279.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0200.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_0911.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_1319.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_1217.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0654.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0438.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/meningioma/Tr-me_0778.jpg',
 '/content/drive/MyDrive/brain-tumor-mri-dataset/Testing/notumor/Te-no_0340.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1537.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_1224.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-gl_1297.jpg', '/content/drive/MyDrive/brain-tumor-mri-dataset/Training/glioma/Tr-glTr_0003.jpg']

Sampling from a set deprecated
 since Pvthon 3.9 and will be removed in a subsequent version.



```
st = extract_labels(selected_images)
```

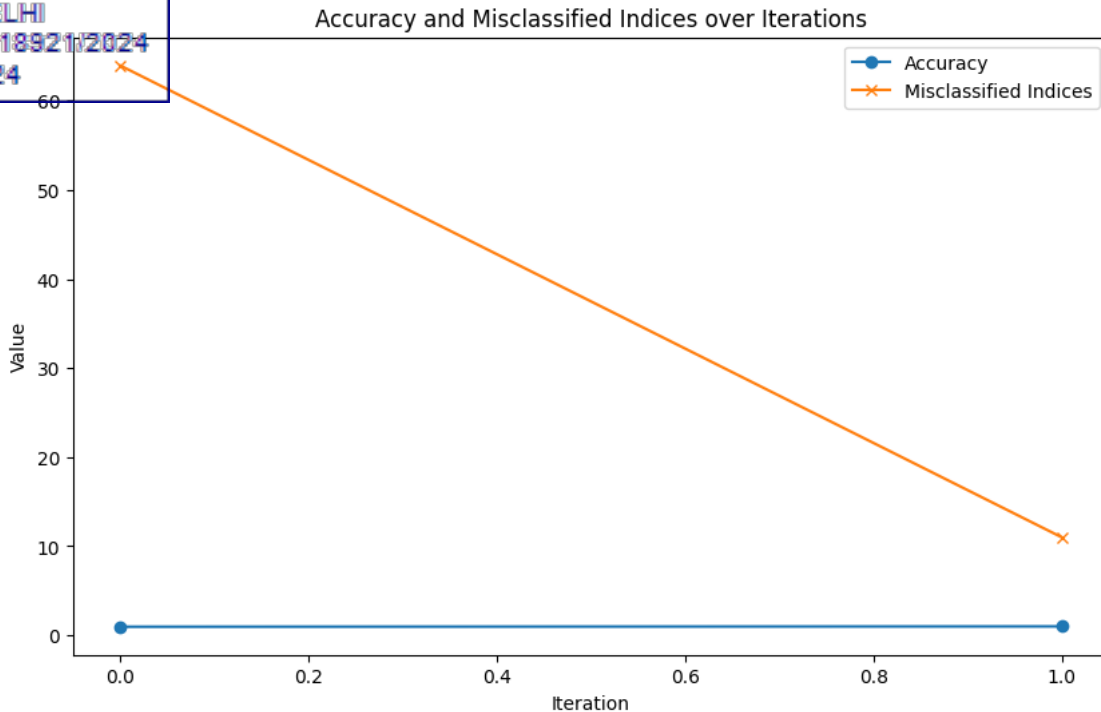
Dr. Anil
 डॉ. अनिल की ओडिओ

```
[ ]: selected_images = label_encoder.fit_transform(selected_images)
final_test = np.array(final_test)
```

```
[ ]: import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(iteration_list, accuracy_list, label='Accuracy', marker='o')
plt.plot(iteration_list, misclassified_list, label='Misclassified Indices',
         marker='x')
plt.xlabel('Iteration')
plt.ylabel('Value')
plt.title('Accuracy and Misclassified Indices over Iterations')
plt.legend()
plt.show()
```

COPYRIGHT OFFICE
NEW DELHI
Reg. No. - SW-18921/2024
Date 03/06/2024



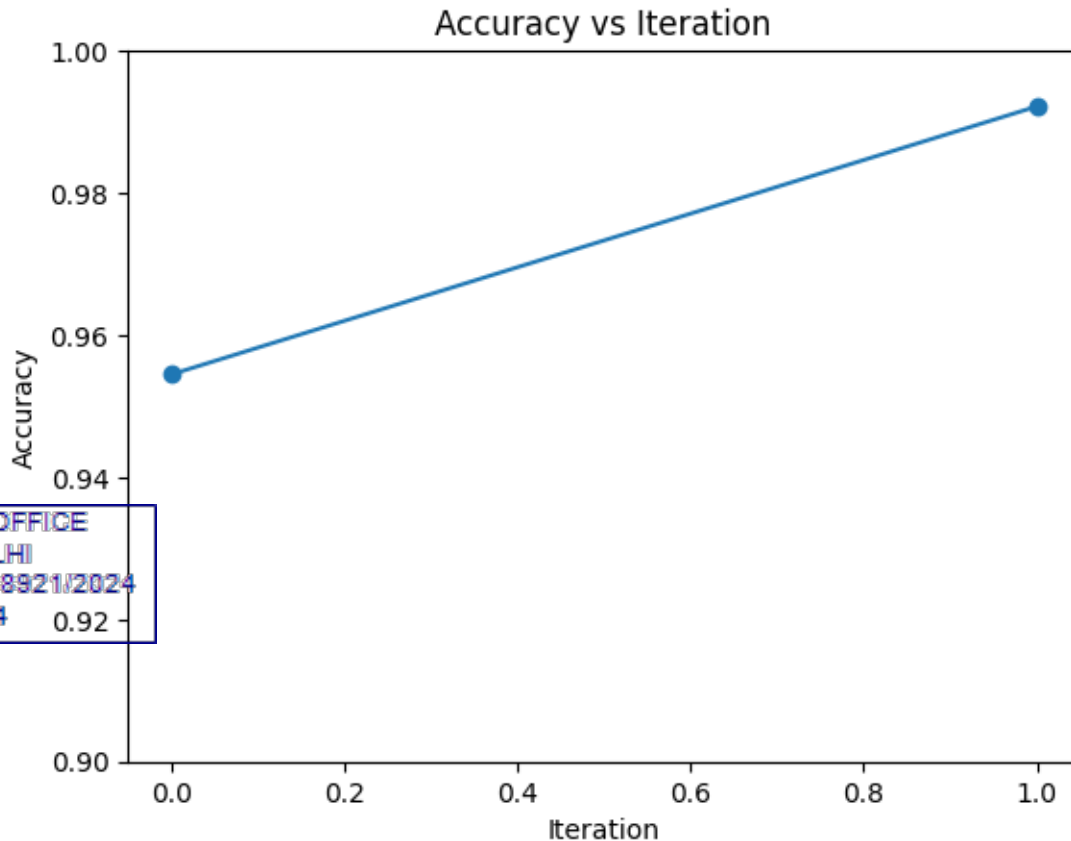
```
[ ]: plt.plot(iteration_list, accuracy_list, marker='o', linestyle='--')
plt.ylim(0.9, 1)

plt.xlabel('Iteration')
plt.ylabel('Accuracy')
plt.title('Accuracy vs Iteration')
```

r()



Dr. Anil
उन्नत की ओर



COPYRIGHT OFFICE
NEW DELHI
Reg. No. - SW-18921/2024
Date 03/06/2024

```
[ ]: print(misclassified_list)
```

```
[64, 11]
```

```
[ ]: from sklearn.metrics import confusion_matrix
import seaborn as sns
predicted_labels = retrained_model.predict(X_test_processed).argmax(axis=1)

conf_matrix = confusion_matrix(Y_test, predicted_labels)

print("Confusion Matrix:")
print(conf_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['notumor', 'pituitary', 'glioma', 'meningioma'],
            yticklabels=['notumor', 'pituitary', 'glioma', 'meningioma'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
```



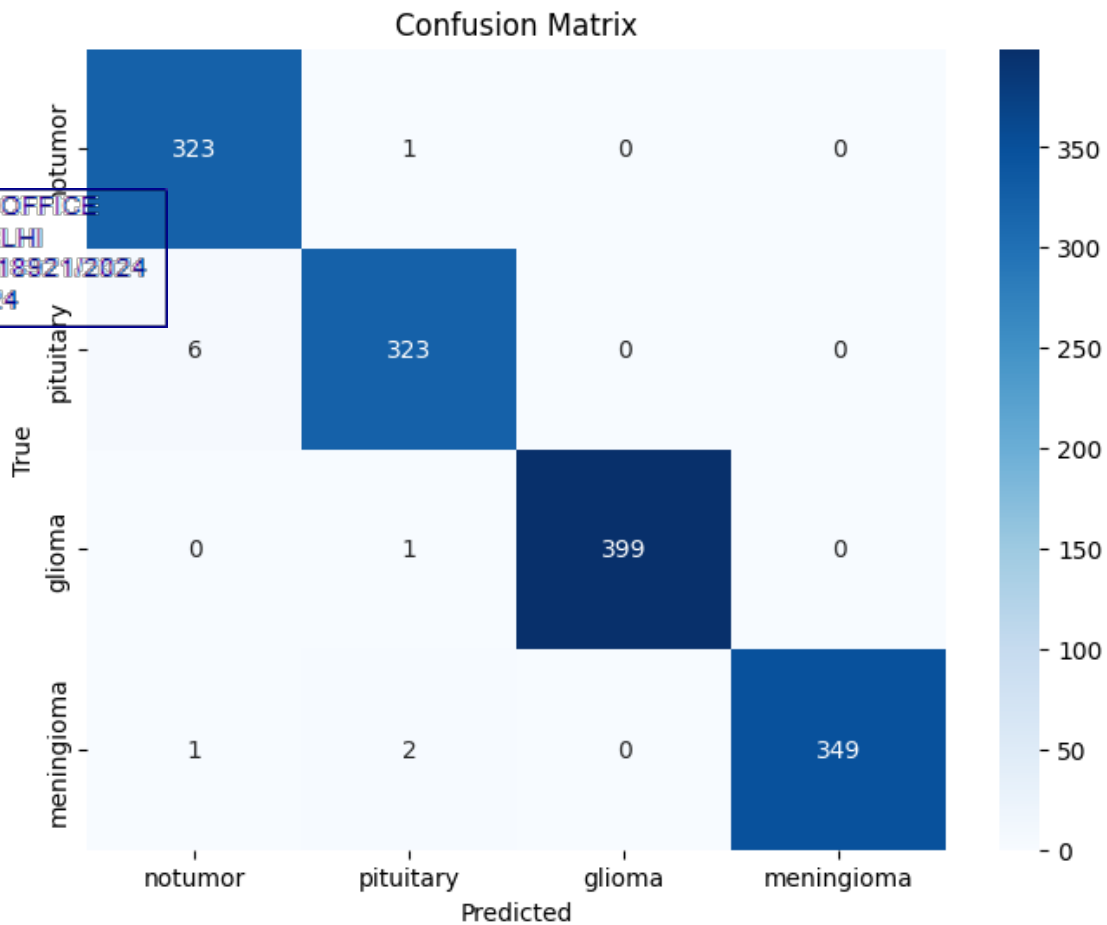
Dr. Anil
उत्कृष्ट की ओर

```
plt.show()
```

44/44 [=====] - 4s 96ms/step

Confusion Matrix:

```
[[323  1  0  0]
 [  6 323  0  0]
 [  0  1 399  0]
 [  1  2  0 349]]
```



COPYRIGHT OFFICE
NEW DELHI
Reg. No. - SW-18921/2024
Date 03/06/2024

```
[ ]: iteration = 0
      accuracy_threshold = 0.98

      while True:
          explainer = shap.Explainer(model, masker, output_names=["notumor",
          ↪ "pituitary", "glioma", "meningioma"])

          misclassified_images = X_test_processed[misclassified_indices]
          shap_values = explainer(misclassified_images, max_evals=500, batch_size=16)
```



Dr. Anil
उन्नत की ओर

```

retrained_model = models.clone_model(model)
retrained_model.set_weights(model.get_weights())

for channel_index in range(X_train_processed.shape[-1]):
    shap_values_channel = shap_values[..., channel_index].values
    base_shap_values_channel = shap_values[..., channel_index].base_values

    positive_shap_mask = base_shap_values_channel > 0
    positive_shap_positions = np.where(positive_shap_mask)

    shap_values_flat = shap_values_channel.flatten()
    shap_values_reshaped = np.reshape(shap_values_flat,
    ↪base_shap_values_channel.shape)

    delta_weight = np.
    sum((base_shap_values_channel[positive_shap_positions] -
    ↪shap_values_reshaped[positive_shap_positions]))

    sample_weights = np.ones_like(Y_train)
    sample_weights[Y_train == channel_index] = 1 + delta_weight

    retrained_model.compile(optimizer=Adam(learning_rate=0.0001),
                            loss=lambda true_labels, predicted_labels:
    ↪custom_loss_with_weights(true_labels, predicted_labels, sample_weights),
                            metrics=['accuracy'])

    retrained_model.fit(X_train_processed, Y_train, epochs=10,
    ↪validation_data=(X_test_processed, Y_test))

    iteration += 1

    test_loss, test_accuracy = retrained_model.evaluate(X_test_processed,
    ↪Y_test)

    predicted_labels = retrained_model.predict(X_test_processed).argmax(axis=1)

    misclassified_indices = [i for i in range(len(Y_test)) if
    ↪predicted_labels[i] != Y_test[i]]

    print(f"Iteration {iteration} - Test Accuracy: {test_accuracy}")

    with open(f'Iteration_{iteration}_model.pkl', 'wb') as file:
        pickle.dump(retrained_model, file)

    test_accuracy >= accuracy_threshold:

```

COPYRIGHT OFFICE
 NEW DELHI
 Reg. No. - SW/18921/2024
 Date 03/05/2024



Dr. Anil
 उन्नत की ओर

```
print(f"Accuracy reached the threshold ({accuracy_threshold}). Stopping  
→training.")  
break
```

COPYRIGHT OFFICE
NEW DELHI
Reg. No. - SW-18921/2024
Date 03/06/2024



Dr. Anil
उत्पादकी अंडिया

```
[ ]: !pip install shap --upgrade
```

Collecting shap

Downloading shap-0.44.1-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (535 kB)

535.7/535.7

kB 9.6 MB/s eta 0:00:00

Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from shap) (1.25.2)

Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from shap) (1.11.4)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from shap) (1.2.2)

Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from shap) (1.5.3)

Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.10/dist-packages (from shap) (4.66.2)

Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.10/dist-packages (from shap) (23.2)

Collecting slicer==0.0.7 (from shap)

Downloading slicer-0.0.7-py3-none-any.whl (14 kB)

Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from shap) (0.58.1)

Requirement already satisfied: cloudpickle in /usr/local/lib/python3.10/dist-packages (from shap) (2.2.1)

Requirement already satisfied: llvmlite<0.42,>=0.41.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba->shap) (0.41.1)

Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2023.4)

Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (1.3.2)

Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (3.3.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.1->pandas->shap) (1.16.0)

Installing collected packages: slicer, shap

Successfully installed shap-0.44.1 slicer-0.0.7

```
[ ]: import shap
import pickle
import matplotlib.pyplot as plt
```

```
n('model.pkl', 'rb') as file:
l = pickle.load(file)
```



Dr. Anil
उन्नत की ओर

```
[ ]: predicted_labels = model.predict(X_test_processed).argmax(axis=1)
```

```
misclassified_indices = []
```

```
for i in range(len(Y_test)):
    if predicted_labels[i] != Y_test[i]:
        misclassified_indices.append(i)
print(misclassified_indices)
```

```
44/44 [=====] - 6s 48ms/step
[53, 70, 74, 148, 157, 172, 194, 198, 204, 225, 251, 259, 298, 320, 335, 375,
424, 433, 463, 482, 483, 536, 537, 550, 566, 602, 617, 622, 624, 631, 666, 667,
694, 705, 718, 723, 728, 736, 755, 758, 760, 784, 802, 814, 872, 930, 945, 985,
1047, 1095, 1170, 1191, 1192, 1200, 1235, 1240, 1263, 1265, 1268, 1287, 1299,
1315, 1391]
```

COPYRIGHT OFFICE

NEW DELHI

Reg. No. - 511/1993/1/2004

Date 03/05/2024

```
misclassified_image_index = misclassified_indices[0]
misclassified_image_path = all_images[misclassified_image_index]
print("Path of misclassified image:", misclassified_image_path)
```

Path of misclassified image: /content/drive/MyDrive/brain-tumor-mri-dataset/Training/notumor/Tr-no_0711.jpg

```
[ ]: misclassified_image = plt.imread(misclassified_image_path)
```

```
[ ]: import cv2
resized_img = cv2.resize(misclassified_image, (224, 224))
resized_imgs=[]
resized_imgs.append(resized_img)
```

```
[ ]: masker = shap.maskers.Image("inpaint_telea", (224,224,3))
misclassified_images = X_test_processed[misclassified_indices]
misclassified_images = np.array(misclassified_images)
misclassified_images = misclassified_images.astype("float32")
explainer = shap.Explainer(model, masker, output_names=["notumor", "pituitary",
↳ "glioma", "meningioma"])
shap_values = explainer(
    misclassified_images[0:1], max_evals=500, batch_size=50, outputs=shap.
↳ Explanation.argsort.flip[:1]
)
```

0% | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 2it [00:14, 14.43s/it]

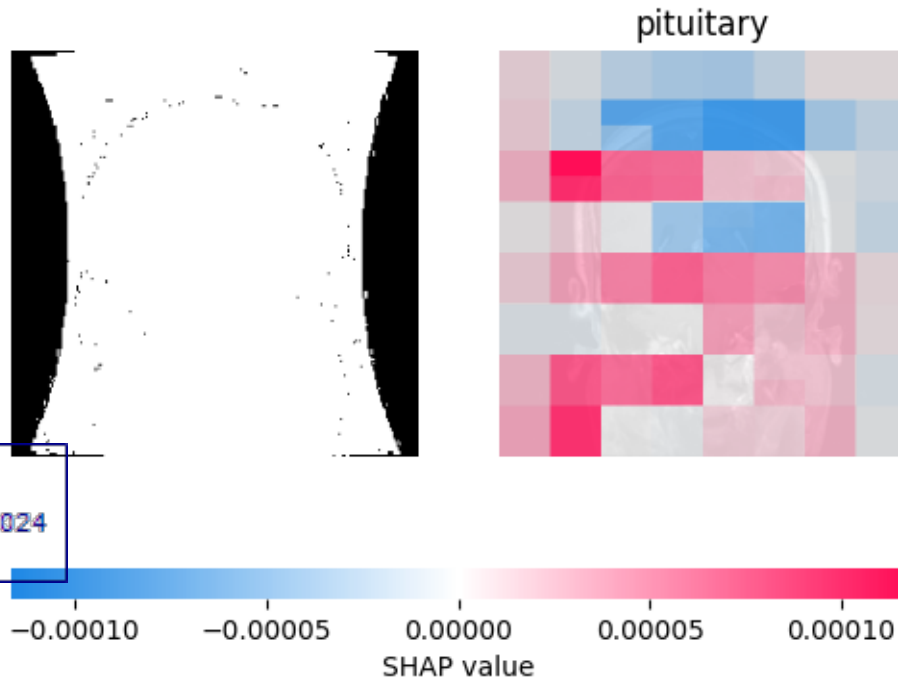
```
.ge_plot(shap_values)
```

matplotlib.image:Clipping input data to the valid range for imshow with

Dr. Ankit
डॉ. अंकित की ओडि



RGB data ([0..1] for floats or [0..255] for integers).



```
[ ]: print(shap_values.shape)
      print(type(shap_values))
```

```
(1, 224, 224, 3, 1)
<class 'shap._explanation.Explanation'>
```

```
[ ]: with open('Iteration_1_model.pkl', 'rb') as file:
      retrained_model = pickle.load(file)
```

```
[ ]: retrained_predicted_labels = retrained_model.predict(X_test_processed).
      ↪argmax(axis=1)
```

44/44 [=====] - 2s 34ms/step

```
[ ]: retrained_misclassified_indices = []

      for i in range(len(Y_test)):
          if retrained_predicted_labels[i] != Y_test[i]:
              retrained_misclassified_indices.append(i)
      print(retrained_misclassified_indices)
```

7, 728, 760, 771, 784, 1102, 1110, 1192, 1316, 1377]



Dr. Anil
उन्नत की ओर

```
[ ]: common_elements = set(misclassified_indices).
      ↳ intersection(retrained_misclassified_indices)
      print(common_elements)
```

{1377, 1316, 1192, 760, 784, 728}

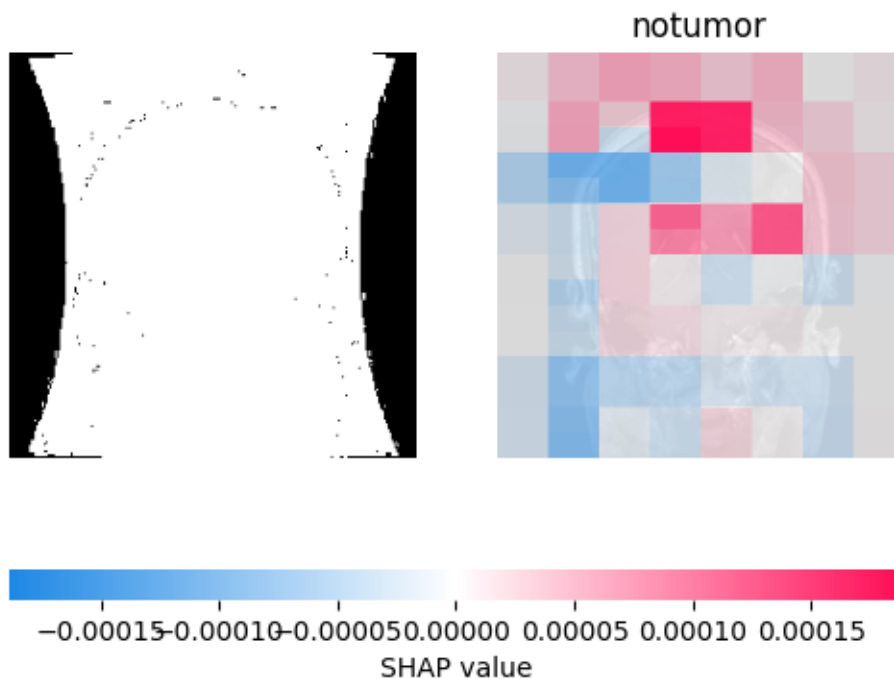
```
[ ]: retrained_explainer = shap.Explainer(retrained_model, masker,
      ↳ output_names=["notumor", "pituitary", "glioma", "meningioma"])
      retrained_shap_values = retrained_explainer(
          misclassified_images[0:1], max_evals=500, batch_size=50, outputs=shap.
          ↳ Explanation.argsort.flip[:1])
      )
```

0% | 0/498 [00:00<?, ?it/s]

PartitionExplainer explainer: 2it [00:16, 16.77s/it]

```
shap_image_plot(retrained_shap_values)
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



Dr. Anil
उन्नत की ओर